PART A — HTML Revision

HTML (HyperText Markup Language) is the skeleton of a webpage. It defines the structure and content using tags. Tags are written inside angle brackets < >, most come in pairs (opening + closing).

1. Basic Page Skeleton

Every HTML file must start with this structure:

```
<!DOCTYPE html>            <!-- Tells browser: this is HTML5 -->
<html lang="en">           <!-- Root element of the page -->
  <head>                  <!-- Meta info (not visible on page) -->
      <meta charset="UTF-8">
    <title>Page Title</title>
      <link rel="stylesheet" href="style.css">
  </head>
  <body>                  <!-- Everything visible goes here -->

  </body>
</html>
```

Note: The <head> tag is like the settings of the page. The <body> tag is what users actually see.

2. Structure & Layout Tags

These tags define the sections and containers of a webpage.

Tag	What it does	Example
** **	Generic block-level container. Used to group elements and apply CSS.	`<div class="box">...</div>`
****	Generic inline container. Used to style a small piece of text inside a line.	`<span style="color:red">hi</span>`
<header>	Top section of the page — usually holds logo, nav, heading.	`<header><h1>My Site</h1></header>`
<nav>	Navigation bar — contains links to different sections/pages.	`<nav><a href="#">Home</a></nav>`

<main>	Main content area of the page. Should appear only once.	`<main>...</main>`
<section>	A thematic grouping of content (e.g. 'About', 'Projects').	`<section id="about">...</section>`
<footer>	Bottom section — copyright, social links, contact info.	`<footer>© 2025</footer>`

3. Text & Heading Tags

These tags deal with displaying text content on the page.

Tag	What it does	Example
<h1> – <h6>	Headings. h1 is biggest, h6 is smallest. Use only ONE h1 per page.	`<h1>Main Title</h1>`
<p>	Paragraph of text. Automatically adds spacing above and below.	`<p>Hello World</p>`
****	Bold text — also tells the browser this text is important.	`<strong>Important!</strong>`
****	Italic text — used for emphasis.	`<em>Note this</em>`
** **	Line break — moves text to the next line. Self-closing tag.	`Line 1 Line 2`
<hr>	Horizontal line (divider). Self-closing tag.	`<hr>`

4. Link & Image Tags

Tag	What it does	Example
<a>	Anchor tag — creates a clickable hyperlink. href attribute holds the URL. target="_blank" opens in new tab.	`<a href="https://google.com" target="_blank">Google</a>`
<img>	Displays an image. src = path to image. alt = text shown if image fails. Self-closing.	`<img src="pic.jpg" alt="photo">`

5. List Tags

Tag	What it does	Example
** **	Unordered list — bullet points.	`<ul><li>Item</li></ul>`
** **	Ordered list — numbered items.	`<ol><li>First</li></ol>`
** **	List item — goes inside or .	`<li>One item</li>`

6. Form Tags

Forms collect user input. Very commonly used with DOM and JavaScript.

Tag	What it does	Example
<form>	Container that wraps all form elements. action sets where data goes, method sets how (GET/POST).	`<form action="" method="post">`
<input>	Most used form element. type attribute changes its behaviour — text, number, email, password, checkbox, radio. Self-closing.	`<input type="text" id="name">`
<label>	Describes an input field. Linked via for attribute matching input's id.	`<label for="name">Name:</label>`
<button>	Clickable button. Used with onclick in JS.	`<button onclick="go()">Submit</button>`
<select>	Dropdown menu container.	`<select id="city">...</select>`
<option>	One choice inside a <select>.	`<option value="del">Delhi</option>`
<textarea>	Multi-line text input box.	`<textarea id="msg"></textarea>`

7. Table Tags

Tables display data in rows and columns.

Tag	What it does	Example
<table>	Outer container that wraps the whole table.	`<table>...</table>`
<thead>	Table header group — wraps the heading row(s).	`<thead><tr>...</tr></thead>`
<tbody>	Table body — wraps the data rows.	`<tbody><tr>...</tr></tbody>`
<tr>	Table row — one horizontal row.	`<tr><td>...</td></tr>`
<th>	Table heading cell — bold and centered by default.	`<th>Name</th>`
<td>	Table data cell — regular cell content.	`<td>Vaibhav</td>`

Note: In DOM class we will add rows to <tbody> dynamically using innerHTML += — so remember these tags!

8. Commonly Used HTML Attributes

Attributes add extra info or behaviour to tags. Written inside the opening tag.

Tag	What it does	Example
id	Unique identifier for ONE element on the page. Used by JS to find it.	`id="userName"`
class	Reusable name. Multiple elements can share a class. Used for CSS styling.	`class="card active"`
href	URL for <a> links.	`href="https://..."`
src	File path or URL for <img> and <script>.	`src="logo.png"`
alt	Alternative text for images — shown if image doesn't load.	`alt="Profile pic"`
type	Sets input type: text, number, email, password, etc.	`type="email"`
placeholder	Grey hint text inside an input (disappears when user types).	`placeholder="Enter name"`

value	Sets the default value of an input or option.	`value="Hello"`
onclick	Runs a JS function when element is clicked.	`onclick="submit()"`
style	Inline CSS applied directly on the element. Avoid overusing it.	`style="color:red"`

PART B — CSS Revision

CSS (Cascading Style Sheets) controls how HTML elements look — colour, size, spacing, layout, and more. Without CSS a page is just plain black-and-white text.

1. CSS Syntax

CSS is written as a set of rules. Each rule targets HTML elements (via a selector) and applies styles (property: value pairs).

```
selector {
  property: value;
  property: value;
}
```

```
/* Example */
h1 {
  color: blue;
  font-size: 32px;
}
```

2. How to Select Elements

Property	Common Values	Description
Tag selector	`p { }  h1 { }`	Selects ALL elements of that tag type.
Class selector	`.card { }  .btn { }`	Selects elements with that class name. Use . (dot) before the name.
ID selector	`#header { }  #box { }`	Selects ONE element with that id. Use # before the name.

`* selector`	`* { }`	Selects every element on the page (often used for reset).

```
/* Selecting by tag */
p { font-size: 16px; }

/* Selecting by class */
.card { background: white; }

/* Selecting by id */
#mainTitle { color: navy; }
```

3. Where to Write CSS

Property	Common Values	Description
`External file`	`<link rel="stylesheet" href="style.css">`	Best practice — write all CSS in a separate .css file and link it in <head>.
`Internal <style>`	`<style> p{} </style>`	Write CSS inside <style> tag in <head>. Good for quick tests.
`Inline style`	`style="color:red"`	Directly on the HTML element. Avoid using too much — hard to maintain.

💡 **Note:** Always prefer an external CSS file for your projects — it keeps HTML clean and makes changes easier.

4. Text & Colour Properties

Property	Common Values	Description
`color`	`red` `#333` `rgb(0,0,255)`	Sets the text colour of an element.
`background-color`	`blue` `#fff` `rgba(0,0,0,0.5)`	Sets the background colour of an element (also written as background: colour).
`font-size`	`16px` `1.5rem` `24pt`	Controls how big the text is. px is most common.
`font-weight`	`normal` `bold` `600`	Thickness of text. bold = 700, normal = 400.
`font-family`	`Arial` `'Times New Roman'` `sans-serif`	Font used for the text. Always provide a fallback font.

text-align	`left center right justify`	Horizontal alignment of text inside an element.
text-decoration	`none underline line-through`	Adds or removes underlines (e.g. remove underline from <a> links).

```
/* Colour & text example */
body {
  background-color: #f4f4f4;  /* light grey background */
  color: #333333;               /* dark text — easy on eyes */
  font-family: Arial, sans-serif;
}

h1 {
  color: #2e75b6;   /* blue heading */
  text-align: center;
}
```

5. Spacing — Margin, Padding, Border

Every HTML element is a box. Understanding this box model is key to controlling spacing and layout.

Property	Common Values	Description
padding	`10px 10px 20px 10px 5px 10px 5px`	Space INSIDE the element, between content and its border. Expands element size.
margin	`10px auto 10px 0`	Space OUTSIDE the element, between it and neighboring elements. margin: auto centres a block.
border	`1px solid #ccc 2px dashed red`	Draws a border around the element. Shorthand: size style colour.
border-radius	`5px 50% 10px 0`	Rounds the corners. 50% makes a circle.
width	`200px 100% 50vw`	Sets the horizontal size of an element.
height	`100px auto 100vh`	Sets the vertical size. auto lets content decide height.

```
/* Box model example */
.card {
  width: 200px;
  padding: 16px;          /* space inside the card */
  margin: 10px;           /* space around the card */
  border: 1px solid #ccc; /* thin grey border */
```

```
  border-radius: 8px;      /* rounded corners */
  background-color: white;
}
```

💡 **Note:** Quick memory trick — Padding = pillow (cushion inside), Margin = moat (space outside).

6. The display Property

display controls how an element behaves in the page layout — whether it sits on its own line, flows inline, or becomes a flex container.

Property	Common Values	Description
`display: block`	`block`	Element takes the full width of its parent and starts on a new line. Default for , <p>, <h1>.
`display: inline`	`inline`	Element sits in the flow of text — no width/height control. Default for , <a>.
`display: inline-block`	`inline-block`	Sits inline like text BUT you can set width and height. Useful for buttons and small boxes.
`display: none`	`none`	Completely hides the element — it takes NO space on the page.
`display: flex`	`flex`	Makes the element a Flex Container — children are arranged using Flexbox rules.

7. Flexbox — display: flex

Flexbox is the easiest way to arrange items side by side or stack them and control their spacing and alignment. Set display: flex on the PARENT/CONTAINER — the children (items inside it) will automatically get arranged by Flexbox.

```
/* The parent becomes a flex container */
.container {
  display: flex;
}

/* Children become flex items automatically */
```

flex-direction — Which direction do items flow?

Property	Common Values	Description
row	`flex-direction: row`	Default. Items sit left to right in a horizontal line →.
row-reverse	`flex-direction: row-reverse`	Items flow right to left ←.
column	`flex-direction: column`	Items stack top to bottom ↓.
column-reverse	`flex-direction: column-reverse`	Items stack bottom to top ↑.

justify-content — How are items spaced along the MAIN axis?

When flex-direction is row, main axis = horizontal. When column, main axis = vertical.

Property	Common Values	Description
flex-start	`justify-content: flex-start`	All items packed to the START (left for row). Default.
flex-end	`justify-content: flex-end`	All items packed to the END (right for row).
center	`justify-content: center`	Items grouped in the centre.
space-between	`justify-content: space-between`	Equal space BETWEEN items, no space at edges. Very common.
space-around	`justify-content: space-around`	Equal space around each item (half space at edges).
space-evenly	`justify-content: space-evenly`	Equal space everywhere including edges.

align-items — How are items aligned on the CROSS axis?

Cross axis is always 90° to the main axis. For row direction, cross axis = vertical.

Property	Common Values	Description
stretch	`align-items: stretch`	Items stretch to fill the container height. Default.
flex-start	`align-items: flex-start`	Items aligned to the top (for row direction).
flex-end	`align-items: flex-end`	Items aligned to the bottom (for row direction).

center	`align-items: center`	Items centred vertically (for row direction). Very useful.

Other useful flex properties

Property	Common Values	Description
flex-wrap	`wrap  nowrap`	wrap: items wrap to next line if container is too narrow. nowrap: forces one line (default).
gap	`10px  10px 20px`	Spacing between flex items. Shorthand for row-gap and column-gap. Much cleaner than using margin.

8. Complete Flexbox Example

A centered card row — the most common layout pattern you will use:

```
/* HTML */
<div class="card-row">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
</div>

/* CSS */
.card-row {
  display: flex;                  /* make it a flex container */
  flex-direction: row;            /* arrange children left to right */
  justify-content: center;        /* centre the group horizontally */
  align-items: center;            /* centre items vertically */
  flex-wrap: wrap;                /* wrap to next line on small screens */
  gap: 16px;                      /* 16px space between cards */
}

.card {
  width: 150px;
  padding: 20px;
  background-color: #2e75b6;
  color: white;
  border-radius: 8px;
  text-align: center;
}
```

💡 **Note:** To perfectly centre ONE item both horizontally and vertically inside a container, use: display:flex + justify-content:center + align-items:center on the parent. This works for full-page centring too!

9. Quick Reference Cheat Sheet

```
/* === TEXT & COLOUR === */
color: #333;                    /* text colour */
background-color: #f4f4f4;      /* background */
font-size: 16px;
font-weight: bold;
font-family: Arial, sans-serif;
text-align: center;

/* === SPACING === */
padding: 10px;                  /* inside space */
margin: 10px auto;              /* outside space | auto centres block */
border: 1px solid #ccc;
border-radius: 8px;

/* === SIZE === */
width: 100%;
height: auto;

/* === FLEXBOX === */
display: flex;
flex-direction: row;            /* or column */
justify-content: center;        /* space along main axis */
align-items: center;            /* space along cross axis */
flex-wrap: wrap;
gap: 16px;

/* === VISIBILITY === */
display: none;                  /* hide element */
display: block;                 /* show as block */
```

💡 **Note:** In our DOM examples today, the container div uses display:flex + flex-wrap:wrap + gap to automatically arrange the movie cards in a neat grid without writing any JS layout code.

JavaScript DOM (Document Object Model) — Class Notes

1. What is the DOM?

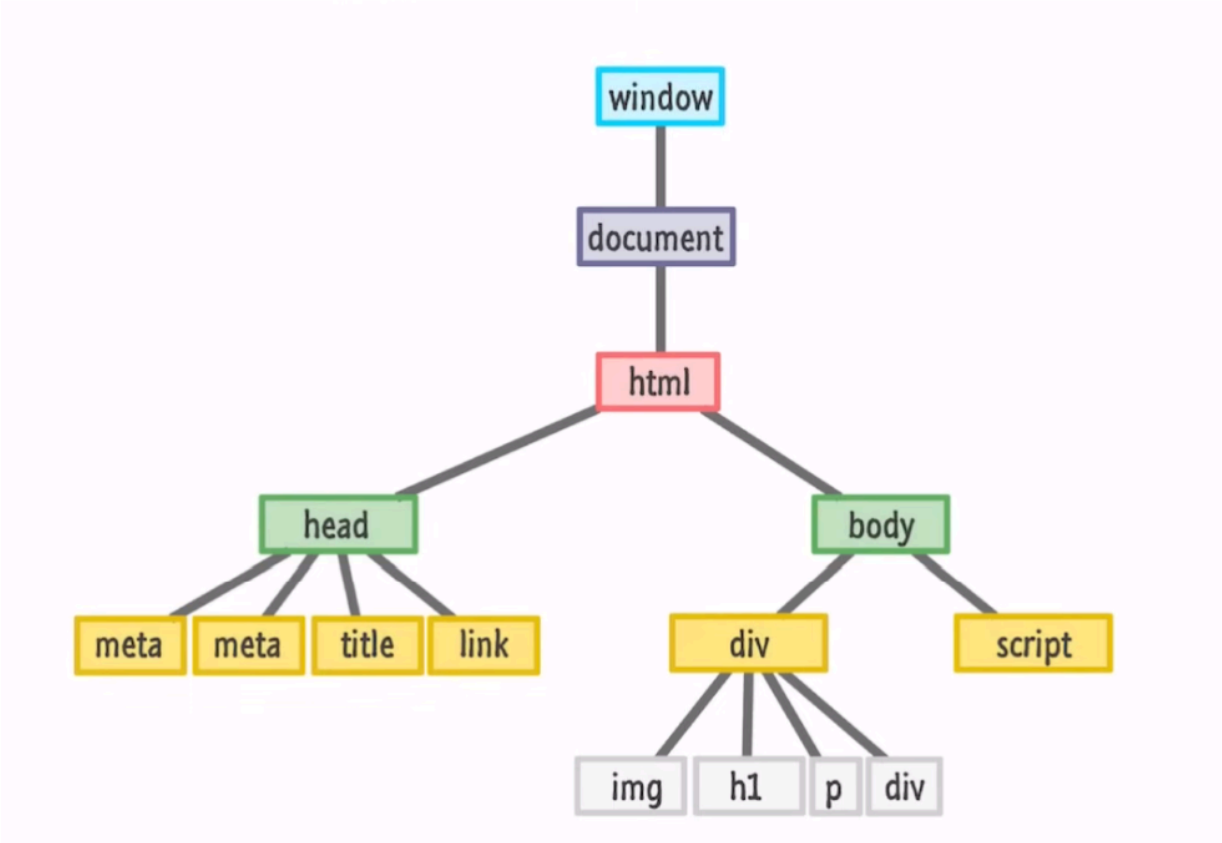
The DOM (Document Object Model) is a programming interface for HTML and XML documents. When a web page loads, the browser creates a DOM — a tree-like structure made of objects — that represents every element on the page (tags, text, attributes).

JavaScript can use the DOM to read and change the content, structure, and style of a web page while it is running, without reloading it. In simple words: HTML gives the page its structure, CSS gives it style, and the DOM (through JS) lets us change that structure and content dynamically.

DOM stands for **Document Object Model**.

When a web page is loaded, the browser converts the HTML document into a **tree-like structure** called the DOM.

Window Object - The window object represents an open window in a web browser. It is a browser's object, not a javascript object and is automatically created by the browser. It is a global object with lots of properties and methods.



Key Points

- **Document object:** "document" is the entry point — it represents the whole HTML page in JS.
- **Tree structure:** Every HTML tag becomes a "node" — nested tags become parent-child nodes.
- **Dynamic:** DOM lets JS add, remove, or change elements live, e.g. after a button click or form submit.

- **Not part of JS language:** DOM is a Web API provided by the browser, JS just talks to it.

Common DOM Methods/Properties Used Today

- document.getElementById("id") → selects one element by its id
- document.querySelector(".class") → selects the first matching element
- document.querySelectorAll(".class") → selects all matching element
- element.value → reads/sets the value typed inside an input box
- element.innerHTML → reads/sets the HTML content inside an element
- element.innerHTML += `...` → appends new HTML to existing content
- element.addEventListener("click", function)→ runs code when an event happens

2. Example 1 — Movie Card Generator

Goal: Take 3 inputs from the user — Movie Image URL, Movie Name, and Rating — and display them as a styled card (div) on the page using innerHTML += , without reloading the page.

HTML (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Movie Card Generator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h1> Movie Card Generator</h1>

  <div class="form-box">
       <input type="text" id="movieUrl" placeholder="Movie Image URL">
       <input type="text" id="movieName" placeholder="Movie Name">
       <input type="number" id="movieRating" placeholder="Rating (out of 10)" min="0"
max="10">
       <button onclick="addMovie()">Add Movie</button>
  </div>

  <div id="movieContainer" class="movie-container"></div>

  <script src="script.js"></script>
</body>
</html>
```

CSS (style.css)

```css
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background: #f4f4f4;
  margin: 0;
  padding: 20px;
}

.form-box {
  margin-bottom: 30px;
}

.form-box input {
  padding: 8px;
  margin: 5px;
  width: 200px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

.form-box button {
  padding: 8px 16px;
  background: #2e75b6;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

.movie-container {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 15px;
}

.movie-card {
  width: 180px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.2);
  overflow: hidden;
}

.movie-card img {
  width: 100%;
  height: 220px;
  object-fit: cover;
}

.movie-card h3 {
  font-size: 16px;
  margin: 8px 0 4px;
}
```

```
.movie-card p {
  margin: 0 0 10px;
  color: #555;
}
```

JavaScript (script.js)

```
function addMovie() {
  // 1. Grab values from the 3 inputs using DOM
  let url = document.getElementById("movieUrl").value;
  let name = document.getElementById("movieName").value;
  let rating = document.getElementById("movieRating").value;

  // 2. Basic check so empty cards are not added
  if (url === "" || name === "" || rating === "") {
    alert("Please fill all 3 fields!");
       return;
  }

  // 3. Select the container div where cards will be shown
  let div = document.getElementById("movieContainer");

  // 4. Append a new movie card using innerHTML +=
  div.innerHTML += `
       <div class="movie-card">
       <img src="${url}" alt="${name}">
      <h3>${name}</h3>
       <p> ${rating} / 10</p>
    </div>
  `;

  // 5. Clear the inputs for the next entry
  document.getElementById("movieUrl").value = "";
  document.getElementById("movieName").value = "";
  document.getElementById("movieRating").value = "";
}
```

- getElementById(...).value reads what the user typed/pasted in each input.
- We use a template literal (backticks ` `) so we can insert variables with ${ }.
- innerHTML += keeps the old cards and adds the new one at the end (if we used =, it would erase previous cards).
- Each new card is just an HTML string injected into the DOM tree — that's DOM manipulation in action.

3. Example 2 — User Info Table

Goal: Take Name, Age, and Email from the user and display each entry as a new row in a HTML table using the DOM.

HTML (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>User Info Table</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h1>👤 User Info Table</h1>

  <div class="form-box">
       <input type="text" id="userName" placeholder="Name">
       <input type="number" id="userAge" placeholder="Age">
       <input type="email" id="userEmail" placeholder="Email">
       <button onclick="addUser()">Add User</button>
  </div>

  <table id="userTable">
    <thead>
      <tr>
        <th>Name</th>
        <th>Age</th>
        <th>Email</th>
      </tr>
    </thead>
       <tbody id="userTableBody">
       <!-- new rows will be added here -->
    </tbody>
  </table>

  <script src="script.js"></script>
</body>
</html>
```

CSS (style.css)

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background: #f4f4f4;
  margin: 0;
  padding: 20px;
}

.form-box input {
  padding: 8px;
  margin: 5px;
  width: 160px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
```

```
.form-box button {
  padding: 8px 16px;
  background: #2e75b6;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

table {
  margin: 20px auto;
  border-collapse: collapse;
  width: 60%;
  background: white;
  box-shadow: 0 2px 6px rgba(0,0,0,0.15);
}

th, td {
  border: 1px solid #ddd;
  padding: 10px;
}

th {
  background: #2e75b6;
  color: white;
}

tr:nth-child(even) {
  background: #f2f2f2;
}
```

JavaScript (script.js)

```
function addUser() {
  // 1. Get values from the 3 inputs
  let name = document.getElementById("userName").value;
  let age = document.getElementById("userAge").value;
  let email = document.getElementById("userEmail").value;

  // 2. Validate
  if (name === "" || age === "" || email === "") {
    alert("Please fill all 3 fields!");
       return;
  }

  // 3. Select the table body
  let tableBody = document.getElementById("userTableBody");

  // 4. Append a new row using innerHTML +=
  tableBody.innerHTML += `
       <tr>
      <td>${name}</td>
      <td>${age}</td>
```

```
      <td>${email}</td>
       </tr>
  `;

  // 5. Clear inputs
  document.getElementById("userName").value = "";
  document.getElementById("userAge").value = "";
  document.getElementById("userEmail").value = "";
}
```

Explanation

- Same pattern as Example 1: read values → validate → build HTML string → inject with innerHTML += .
- Here we add a <tr> row instead of a card — showing students the DOM doesn't care what tag it is, the logic is identical.
- tbody is targeted (not the whole table) so the header row never gets overwritten.

Quick Recap

- DOM = live tree of the page that JS can read and change.
- Step pattern used in both examples: Select inputs → Read .value → Validate → Build HTML with template literals → Inject with innerHTML += → Clear inputs.
- innerHTML = replaces everything; innerHTML += adds to the end.
- The same technique works for any data: movies, users, products, comments, todo items, etc.

Introduction to JavaScript

JavaScript is a versatile, dynamically typed programming language that brings life to web pages by making them interactive. It is used for building interactive web applications and supports both client-side and server-side development.

- **Interpreted language:** Code is executed line by line.
- **Dynamically typed:** Variable types are determined at runtime.
- **Single-threaded:** Executes one task at a time (but supports asynchronous operations).

Features of JavaScript

Here are some key features of JavaScript that make it a powerful language for web development:

- **Client-Side Scripting:** JavaScript runs on the user's browser, so it has a faster response time without needing to communicate with the server .
- **Versatile:** Can be used for a wide range of tasks, from simple calculations to complex server-side applications.
- **Event-Driven:** Responds to user actions (clicks, keystrokes) in real-time.
- **Asynchronous:** It can handle tasks like fetching data from servers without freezing the user interface.
- **Rich Ecosystem:** There are numerous libraries and frameworks built on JavaScript, such as React , Angular , and Vue.js, which make development faster and more efficient.

1. Variables

A variable is a named container used to store data values in memory. In JavaScript, variables are declared using var, let, or const.

- var — function-scoped, can be re-declared and updated, hoisted with a default value of undefined.
- let — block-scoped, can be updated but not re-declared in the same scope, hoisted but not initialized.
- const — block-scoped, cannot be updated or re-declared; must be initialized at the time of declaration.

```
var age = 21;              // function-scoped
let studentName = "Riya";  // block-scoped, can change
const college = "ABC Engineering College"; // cannot change

studentName = "Riya Sharma"; // OK
// college = "XYZ College"; // Error: Assignment to constant variable
```

2. Data Types

JavaScript has two broad categories of data types: Primitive types and Non-Primitive (Reference) types.

Primitive Data Types

- Number — e.g. 10, 3.14
- String — e.g. "Hello", 'JavaScript'
- Boolean — true or false
- Undefined — a variable declared but not assigned a value
- Null — represents an intentional absence of value
- Symbol — a unique and immutable value
- BigInt — for numbers larger than the safe integer limit

Non-Primitive (Reference) Data Types

- Object — collection of key-value pairs
- Array — ordered list of values
- Function — a reusable block of code

```
let rollNo = 45;                // Number
let name = "Aman";              // String
let isPresent = true;           // Boolean
let marks;                      // Undefined
let result = null;              // Null

let student = { name: "Aman", rollNo: 45 }; // Object
let subjects = ["Maths", "DBMS", "OS"];   // Array

console.log(typeof rollNo);   // "number"
console.log(typeof name);     // "string"
```

```
console.log(typeof student);  // "object"
```

3. if...else Statement

The if...else statement is used to perform different actions based on different conditions.

```
let marks = 72;

if (marks >= 90) {
  console.log("Grade: A+");
} else if (marks >= 75) {
  console.log("Grade: A");
} else if (marks >= 60) {
  console.log("Grade: B");
} else {
  console.log("Grade: C");
}
// Output: Grade: B
```

4. for Loop

The for loop is used to execute a block of code repeatedly, a specific number of times.

```
for (let i = 1; i <= 5; i++) {
  console.log("Roll No: " + i);
}
// Output:
// Roll No: 1
// Roll No: 2
// Roll No: 3
// Roll No: 4
// Roll No: 5
```

Syntax breakdown: for (initialization; condition; increment/decrement) { ...code... }

5. Functions

A function is a reusable block of code designed to perform a particular task. It runs only when it is called (invoked).

```
function greetStudent(name) {
  return "Hello, " + name + "! Welcome to class.";
}

console.log(greetStudent("Priya"));
// Output: Hello, Priya! Welcome to class.
```

- Function Declaration — defined using the function keyword, hoisted completely.
- Function Expression — a function assigned to a variable, not hoisted the same way.

```
// Function Expression
const add = function (a, b) {
  return a + b;
};
console.log(add(4, 5)); // 9
```

6. Arrow Functions

Arrow functions, introduced in ES6, provide a shorter syntax for writing functions.

```
// Normal function expression
const square = function (n) {
  return n * n;
};

// Arrow function equivalent
const squareArrow = (n) => {
  return n * n;
};

// Shorter arrow function (implicit return)
const squareShort = n => n * n;

console.log(squareShort(6)); // 36
```

Arrow Functions allow a shorter syntax for **function expressions**.

You can skip the **function** keyword, the **return** keyword, and the **curly brackets**:

const multiply = (a, b) => a * b;

Example 1: Arrow Function Without Parameters

```
const greet = () => {
    console.log("Hello, Welcome to JavaScript!");
};

greet();
```

Output:

```
Hello, Welcome to JavaScript!
```

Example 2: Arrow Function with One Parameter

```
const square = (num) => {
    return num * num;
};

console.log(square(5));
```

Output:

```
25
```

Example 3: Arrow Function with Multiple Parameters

```
const add = (a, b) => {
    return a + b;
};

console.log(add(10, 20));
```

Output:

```
30
```

Higher Order Functions -

A **Higher Order Function** is a function that:

1. **Takes another function as an argument**, or
2. **Returns another function.**

Functions like **forEach(), map(), filter(), reduce(), and sort()** are higher-order functions because they accept a callback function.

Suppose we have the following array:

```
const numbers = [10, 20, 30, 40, 50];
```

1. forEach()

Purpose

Used to **perform an operation on every element** of an array.

- Does **not** return a new array.
- Mostly used for printing, updating values, or side effects.

Syntax

```
array.forEach(function(element, index) {

});
```

or

```
array.forEach((element, index) => {

});
```

Example 1

```
const numbers = [10, 20, 30, 40];

numbers.forEach((num) => {
   console.log(num);
});
```

Output

10
20
30
40

Example 2

Print index and value

```
const fruits = ["Apple", "Mango", "Orange"];

fruits.forEach((fruit, index) => {
    console.log(index + " : " + fruit);
});
```

Output

0 : Apple
1 : Mango
2 : Orange

2. map() -

Creates a **new array** after modifying every element.

The original array remains unchanged.

Syntax

```
const newArray = array.map((element) => {

});
```

Example

Multiply every number by 2.

```
const numbers = [10,20,30,40];
```

```
const result = numbers.map((num)=>{
   return num*2;
});

console.log(result);
```

Output

[20,40,60,80]

Original array - [10,20,30,40]

New array - [20,40,60,80]

Example

Convert names to uppercase.

```
const names = ["ekta","rahul","amit"];

const upper = names.map(name => name.toUpperCase());

console.log(upper);
```

Output

["EKTA","RAHUL","AMIT"]

3. filter()

Returns only those elements that satisfy a condition.

Returns a **new array**.

Syntax

```
const ans = array.filter((element)=>{

});
```

Example

Print even numbers.

```
const numbers = [1,2,3,4,5,6];

const even = numbers.filter((num)=>{
   return num%2==0;
});

console.log(even);
```

Output - [2,4,6]

Example

Students having marks greater than 80.

```
const marks = [45,90,67,81,99];

const topper = marks.filter(mark => mark>80);

console.log(topper);
```

Output

[90,81,99]

4. reduce()

Converts the entire array into a **single value**.

Can calculate

- Sum
- Product
- Maximum
- Minimum
- Count

Syntax

```
const result = array.reduce((accumulator,current)=>{
```

},initialValue);

Where

- accumulator → stores answer
- current → current element

Example 1

Find sum.

```
const numbers = [10,20,30,40];

const sum = numbers.reduce((acc,current)=>{
   return acc+current;
},0);

console.log(sum);
```

Output

100

Dry Run

Array

[10,20,30]

Initial accumulator = 0

Iteration 1

acc = 0
current =10

0+10=10

Iteration 2

acc =10
current =20

10+20=30

Iteration 3

acc =30
current =30

30+30=60

Iteration 4

acc = 60
current = 40

60+40 = 100

OUTPUT = 100

Example 2

Largest element

```
const numbers = [10,40,15,80,30];

const max = numbers.reduce((acc,current)=>{
   return current>acc ? current : acc;
},numbers[0]);

console.log(max);
```

Output

80

5. sort()

Sorts the array.

By default, JavaScript sorts elements as **strings (lexicographically)**.

Example

```
const fruits = ["Banana","Apple","Orange"];

fruits.sort();

console.log(fruits);
```

Output

```
["Apple","Banana","Orange"]
```

Problem with Numbers

```
const numbers = [5,100,20];

numbers.sort();

console.log(numbers);
```

Output

```
[100,20,5]
```

Why?

Because JavaScript compares strings.

"100"

"20"

"5"
It is comparing 1,2 and 5 (first digit)

Ascending Order

```
const numbers = [5,100,20,1];

numbers.sort((a,b)=>{
   return a-b;
});

console.log(numbers);
```

Output

[1,5,20,100]

Descending Order

```
const numbers = [5,100,20,1];

numbers.sort((a,b)=>{
   return b-a;
});

console.log(numbers);
```

Output

[100,20,5,1]

Rule

If a-b < 0 (Negative) - no swapping

➡ Keep **a before b**

If a-b > 0 (Positive) - swapping

➡ Put **b before a**

If a-b == 0

➡ Keep their order.

Practice Question -

Suppose we have student objects.

```
const students = [
   {name:"Ekta", marks:90},
   {name:"Rahul", marks:75},
   {name:"Aman", marks:85},
   {name:"Priya", marks:60}
```

```
];
```

Get student names (`map`)

```
const names = students.map(student => student.name);

console.log(names);
```

Output

```
["Ekta", "Rahul", "Aman", "Priya"]
```

Students scoring above 80 (`filter`)

```
const toppers = students.filter(student => student.marks > 80);

console.log(toppers);
```

Output

```
[
  { name: "Ekta", marks: 90 },
  { name: "Aman", marks: 85 }
]
```

Total marks (`reduce`)

```
const total = students.reduce((sum, student) => sum + student.marks, 0);

console.log(total);
```

Output

```
310
```

Sort students by marks (`sort`)

```
students.sort((a, b) => b.marks - a.marks);

console.log(students);
```

Output

```
[
```

```
  { name: "Ekta", marks: 90 },
  { name: "Aman", marks: 85 },
  { name: "Rahul", marks: 75 },
  { name: "Priya", marks: 60 }
]
```

DOM Manipulation -

DOM manipulation is the process of using a scripting language (almost exclusively **JavaScript**) to dynamically select, modify, add, or delete elements and styles within a web document.

index.html > ...

```html
<!DOCTYPE html>
<html>
<head><title>DOM Demo</title></head>
<body>
    <h1 id="title">Welcome to JavaScript</h1>
    <div id="box">
        <p>Paragraph 1</p>
        <p>Paragraph 2</p>

    </div>
    <script src="script.js"></script>
</body>
</html>
```

```javascript
// 1. Selecting Elements

// Select the h1 element using its id
let title = document.getElementById("title");

// Select the div using its id
let box = document.getElementById("box");
```

```
// Print both elements in the console
console.log(title);
console.log(box);


// 2. DOM Manipulation

// Change the text written inside the h1
title.innerText = "DOM Manipulation Demo";


// 3. Changing Styles

// Change text color
title.style.color = "blue";

// Change background color
title.style.backgroundColor = "yellow";

// Increase font size
title.style.fontSize = "35px";

// Add some space inside the element
title.style.padding = "10px";

// Add a border
title.style.border = "2px solid black";

// Add some styles to the div
box.style.border = "2px solid red";
box.style.padding = "15px";
box.style.marginTop = "20px";
box.style.backgroundColor = "lightgray";


// 4. Creating a New Element

// Create a new paragraph element
let newPara = document.createElement("p");
```

```
// Add text inside the new paragraph
newPara.innerText = "I am a new paragraph created using JavaScript.";

// Add the paragraph inside the div
box.appendChild(newPara);

// Now the paragraph is visible on the webpage


// 5. Removing an Element

// Select the first paragraph inside the div
let firstPara = box.firstElementChild;

// Remove it from the webpage
firstPara.remove();


// 6. DOM Traversal

// parentElement -> Gives the parent of an element
console.log("Parent of Title:");
console.log(title.parentElement);

// children -> Gives all child elements
console.log("Children of Box:");
console.log(box.children);

// firstElementChild -> First child inside the div
console.log("First Child:");
console.log(box.firstElementChild);

// lastElementChild -> Last child inside the div
console.log("Last Child:");
console.log(box.lastElementChild);

// previousElementSibling -> Element before the current element
console.log("Previous Sibling of Last Child:");
console.log(box.lastElementChild.previousElementSibling);
```

```
// nextElementSibling -> Element after the first child
console.log("Next Sibling of First Child:");
console.log(box.firstElementChild.nextElementSibling);
```

innerText-

- Used to **read or change the text** of an element.

title.innerText = "DOM Manipulation Demo";

Output:

DOM Manipulation Demo

Changing Styles

Use the **style** property to change CSS using JavaScript.

title.style.color = "blue";
title.style.backgroundColor = "yellow";
title.style.fontSize = "35px";

Some common style properties:

- `color`
- `backgroundColor`
- `fontSize`
- `padding`
- `margin`
- `border`

Creating Elements

Step 1: Create a new element

let newPara = document.createElement("p");

Step 2: Add text

newPara.innerText = "Hello";

Step 3: Add it to the webpage

box.appendChild(newPara);

Create → Add Content → Append

Removing Elements -

Select the element and use `remove()`.

firstPara.remove();

The element is deleted from the webpage.

DOM Traversal -

Traversal means **moving from one element to another**.

`parentElement`

Returns the parent element.

title.parentElement

`children`

Returns all child elements.

box.children

`firstElementChild`

Returns the first child.

box.firstElementChild

`lastElementChild`

Returns the last child.

box.lastElementChild

Event Handling in JavaScript

An **event** is an action performed by the user or browser.

Examples:

- Clicking a button
- Typing on keyboard
- Moving mouse
- Submitting a form
- Loading a webpage

JavaScript listens for these events and performs actions.

Ways to Handle Events -

1. Inline Event Handling

```
<button onclick="showMessage()">Click Me</button>

<script>
function showMessage(){
   alert("Button Clicked");
}
</script>
```

Not recommended for large projects.

2. Using JavaScript Property

```
const btn = document.getElementById("btn");

btn.onclick = function(){
   alert("Button Clicked");
}
```

3. Using addEventListener()

Syntax -

```
element.addEventListener("eventName", function);
```

Example -

```
const btn = document.getElementById("btn");

btn.addEventListener("click", function(){
   alert("Hello");
});
```

Advantages -

- Can add multiple events
- Cleaner code
- Most commonly used

Mouse Events -

Mouse events occur when the user interacts using the mouse.

Event	Description
click	Single click
dblclick	Double click
mouseover	Mouse enters element
mouseout	Mouse leaves element
mousemove	Mouse moves
contextmenu	Right click

click Event

```
<button id="btn">Click</button>

<script>
const btn = document.getElementById("btn");

btn.addEventListener("click", function(){
   alert("Button Clicked");
});
</script>
```

mouseover Event

```
const box = document.getElementById("box");

box.addEventListener("mouseover", function(){
   box.style.backgroundColor = "yellow";
});
```

mouseout Event

```
box.addEventListener("mouseout", function(){
   box.style.backgroundColor = "white";
});
```

dblclick Event

```
btn.addEventListener("dblclick", function(){
   alert("Double Clicked");
});
```

Keyboard Events

Keyboard events occur when the user presses keys.

Event	Description
keydown	Key is pressed
keyup	Key is released
keypress	Older event (avoid using)

keydown Example

```
<input type="text" id="name">

<script>
const input = document.getElementById("name");
```

```
input.addEventListener("keydown", function(){
   console.log("Key Pressed");
});
</script>
```

keyup Example

```
input.addEventListener("keyup", function(){
   console.log(input.value);
});
```

Useful for:

- Live Search
- Password strength
- Character count

Form Events -

Event	Description
submit	Form submitted
change	Value changes
input	Every character typed
focus	Input gets focus
blur	Input loses focus
reset	Form reset

submit Event

```
<form id="myForm">
   <button>Submit</button>
</form>

<script>
```

```
const form = document.getElementById("myForm");

form.addEventListener("submit", function(){
   alert("Form Submitted");
});
</script>
```

input Event

Runs whenever user types.

```
input.addEventListener("input", function(){
   console.log(input.value);
});
```

change Event

Runs after value changes and user leaves the field.

```
input.addEventListener("change", function(){
   console.log("Changed");
});
```

focus Event

```
input.addEventListener("focus", function(){
   input.style.backgroundColor = "lightyellow";
});
```

blur Event

```
input.addEventListener("blur", function(){
   input.style.backgroundColor = "white";
});
```

Event Object -

Whenever an event occurs, JavaScript automatically creates an **Event Object**.

It stores information about the event.

Syntax

element.addEventListener("click", function(event){

});

Common Properties -

Property	Description
event.target	Element that triggered the event
event.type	Event name
event.key	Pressed key
event.preventDefault()	Stops default action

Example

```
btn.addEventListener("click", function(event){
   console.log(event.target);
   console.log(event.type);
});
```

Output

<button>Click</button>

click

Keyboard Event Object

```
input.addEventListener("keydown", function(event){
   console.log(event.key);
});
```

If user presses A

Output

A

Mouse Position

```
document.addEventListener("click", function(event){
    console.log(event.clientX);
    console.log(event.clientY);
});
```

preventDefault()

Some HTML elements have default behavior.

Examples:

- Form submits
- Link opens another page
- Checkbox checks itself

`preventDefault()` stops this default behavior.

Without preventDefault()

<form>

<button>Submit</button>

</form>

Clicking Submit reloads the page.

With preventDefault()

```
const form = document.getElementById("myForm");
```

```
form.addEventListener("submit", function(event){

    event.preventDefault();

    alert("Form Submitted Successfully");

});
```

Now the page will not reload.

Link Example

```
<a href="https://google.com" id="link">
Google
</a>
const link = document.getElementById("link");

link.addEventListener("click", function(event){

    event.preventDefault();

    alert("Opening is blocked");

});
```

Accessing Form Data -

HTML -

```
<input type="text" id="username">
```

JavaScript -

```
const username = document.getElementById("username");

console.log(username.value);
```

`.value` returns user input.

Example

```
const btn = document.getElementById("btn");

btn.addEventListener("click", function(){

    console.log(username.value);

});
```

If user enters

Ekta

Output

Ekta

Form Validation -

Validation means checking whether user input is correct before submitting.

Examples

- Name should not be empty
- Password minimum 8 characters
- Email should be valid
- Age must be positive

Importance of Form Validation -

- Prevent wrong data
- Improve user experience
- Reduce errors
- Improve security (client-side validation is helpful but should always be backed by server-side validation)

Required Field Validation

```
form.addEventListener("submit", function(event){

    event.preventDefault();

    if(username.value === ""){

        alert("Name is required");

    }

});
```

Password Validation

```
if(password.value.length < 8){

    alert("Password should be at least 8 characters");

}
```

Email Validation

```
if(!email.value.includes("@")){

    alert("Invalid Email");

}
```

User Input Handling -

User input can be handled using event listeners.

Example

```
input.addEventListener("input", function(){

    console.log(input.value);
```

});

As user types

H
He
Hel
Hell
Hello

Console

H
He
Hel
Hell
Hello

Displaying Error Messages

Instead of using `alert()`,we can also show errors below the input field.

HTML -

<input type="text" id="username">

JavaScript -

const error = document.getElementById("error");

if(username.value === ""){

    error.textContent = "Name is required";

}

Clearing Error Messages

if(username.value !== ""){

```
    error.textContent = "";

}
```

Example -

HTML

```
<form id="myForm">

<input type="text" id="username">

<p id="error"></p>

<button>Submit</button>

</form>
```

JavaScript

```
const form = document.getElementById("myForm");
const username = document.getElementById("username");
const error = document.getElementById("error");

form.addEventListener("submit", function(event){

    event.preventDefault();

    if(username.value.trim() === ""){

        error.textContent = "Name cannot be empty";

    }
    else{

        error.textContent = "";

        alert("Form Submitted Successfully");

    }

});
```

Local Storage, Session Storage & Browser Data Management in JavaScript

Browser Data Management -

Browser Data Management is the process of **storing, retrieving, updating, and deleting data** in the user's web browser using JavaScript.

Instead of asking the server for the same information every time, websites can store some data directly in the browser.

Examples

- Username
- Theme (Dark/Light Mode)
- Shopping Cart
- Language Preference
- Form Data
- Game Score

Why Do We Need Browser Storage?

Without browser storage:

- User preferences are lost after refreshing the page.
- Users may need to enter the same information repeatedly.
- Every request may have to go to the server.

With browser storage:

- Faster user experience.
- Less communication with the server.
- Data can remain available even after refreshing the page.

Types of Browser Storage

JavaScript mainly provides three ways to store data:

1. Local Storage
2. Session Storage
3. Cookies

In this chapter, we focus on **Local Storage** and **Session Storage**.

Local Storage -

Local Storage is a browser storage mechanism that stores data permanently until it is manually removed.

The stored data remains available even if:

- The page is refreshed.
- The browser is closed.
- The computer is restarted.

The data is deleted only when:

- JavaScript removes it.
- The user clears browser data.

Features of Local Storage

- Stores data as **key-value pairs**.
- Stores only **strings**.
- Capacity is approximately **5–10 MB**.
- Data persists until removed.
- Accessible from all tabs of the same website (same origin).

Syntax

Store Data

localStorage.setItem("key", "value");

Example

localStorage.setItem("name", "Ekta");

Retrieve Data

localStorage.getItem("key");

Example

```
let name = localStorage.getItem("name");
console.log(name);
```

Output

Ekta

Remove One Item

```
localStorage.removeItem("name");
```

Remove All Data

```
localStorage.clear();
```

Example

```
localStorage.setItem("city", "Lucknow");

console.log(localStorage.getItem("city"));
```

Output

Lucknow

Session Storage-

Session Storage stores data only for the current browser tab.

The data remains available while the tab is open.

The data is automatically deleted when the tab is closed.

Refreshing the page **does not remove** Session Storage data.

Features of Session Storage

- Stores key-value pairs.
- Stores only strings.
- Capacity is approximately **5 MB**.
- Exists only while the current tab is open.
- Not shared with other tabs.

Syntax

Store Data

sessionStorage.setItem("key", "value");

Example

sessionStorage.setItem("user", "Ekta");

Retrieve Data

sessionStorage.getItem("user");

Remove One Item

sessionStorage.removeItem("user");

Remove All Data

sessionStorage.clear();

Example

sessionStorage.setItem("course", "Java");

console.log(sessionStorage.getItem("course"));

Output

Java

Close the browser tab:

Data is automatically deleted.

Browser Storage Methods

1. setItem()

Stores data using a key and value.

Syntax

storage.setItem("key", "value");

Example

localStorage.setItem("language", "English");

2. getItem()

Retrieves stored data.

Syntax

storage.getItem("key");

Example

localStorage.getItem("language");

Output

English

3. removeItem()

Removes only the specified key.

Syntax

storage.removeItem("key");

Example

```
localStorage.removeItem("language");
```

4. clear()

Removes every item from the storage.

Syntax

```
storage.clear();
```

Example

```
sessionStorage.clear();
```

5. key()

Returns the key at the specified index.

Syntax

```
storage.key(index);
```

Example

```
localStorage.setItem("name", "Ekta");
localStorage.setItem("city", "Lucknow");

console.log(localStorage.key(0));
console.log(localStorage.key(1));
```

Possible Output

```
name
city
```

6. length

Returns the total number of stored items.

Example

```
localStorage.setItem("name", "Ekta");
localStorage.setItem("city", "Lucknow");

console.log(localStorage.length);
```

Output

2

---

7. How JavaScript Code Runs — Call Stack & Execution Context

JavaScript is a single-threaded, synchronous language. It executes code inside an Execution Context, and every execution context is managed using a Call Stack.

7.1 Call Stack

The Call Stack is a data structure that keeps track of function calls and execution contexts in a JavaScript program. It works on the LIFO (Last In, First Out) principle.

As soon as a JavaScript program starts running, a Global Execution Context (GEC) is created and pushed onto the Call Stack.

7.2 Global Execution Context (GEC)

The Global Execution Context is the base/default context created when the JS engine starts executing a script. It is created in two phases:

- Memory Creation Phase (also called the Creation Phase)
- Code Execution Phase

7.3 Memory Creation Phase

In this phase, the JS engine scans the entire code and allocates memory to variables and functions, without actually running any code:

- Variables (var) are allocated memory and initialized with a special value: undefined.

- let and const variables are also allocated memory, but they remain in an uninitialized state (Temporal Dead Zone) until their actual line of code runs.
- Function declarations are stored in memory with their entire function code (fully hoisted).

7.4 Code Execution Phase

In this phase, the JS engine runs the program line by line, executing statements and assigning actual values to variables that were set to undefined in the memory phase.

Example to understand both phases:

```
var x = 10;
function greet() {
  console.log("Hello!");
}
var y = 20;
greet();
```

Memory Creation Phase (before execution starts):

- x → undefined
- y → undefined
- greet → entire function stored in memory

Code Execution Phase (line by line):

- Line 1: x is assigned the value 10
- Line 2-4: function is already in memory, nothing new happens
- Line 5: y is assigned the value 20
- Line 6: greet() is called → a new Function Execution Context is created and pushed onto the Call Stack, code inside runs, logs "Hello!", then this context is popped off the stack

7.5 Function Execution Context

Every time a function is invoked, a new Function Execution Context (FEC) is created and pushed on top of the Call Stack. It also has its own Memory Creation Phase and Code Execution Phase for the variables and parameters local to that function. Once the function finishes executing, its execution context is popped off the stack, and control returns to where it was called from.

8. Hoisting

Hoisting is JavaScript's default behavior of moving declarations (of variables and functions) to the top of their scope, during the Memory Creation Phase, before the code is executed.

```
console.log(a);   // undefined (not an error)
var a = 5;
console.log(a);   // 5

sayHi();          // works fine, prints "Hi!"
function sayHi() {
  console.log("Hi!");
}
```

var declarations are hoisted and initialized with undefined, which is why accessing them before their line of code does not throw an error. Function declarations are hoisted completely, including their body, so they can be called before they appear in the code.

9. Temporal Dead Zone (TDZ)

The Temporal Dead Zone is the time period between the start of a scope (where a let or const variable is hoisted into memory) and the line where it is actually declared/initialized. During this period, the variable exists but cannot be accessed.

```
console.log(b);  // ReferenceError: Cannot access 'b' before initialization
let b = 10;

console.log(c);  // ReferenceError: Cannot access 'c' before initialization
const c = 20;
```

Unlike var, let and const are hoisted but not initialized with undefined. They stay in the Temporal Dead Zone from the start of the block until their declaration is executed, so trying to use them earlier throws a ReferenceError instead of returning undefined.